Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»
КАФЕДРА ИУ7 «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

Разработка базы данных для поиска и организации досуга

Студент	ИУ7-64Б		Козин М.В.
	(группа)	(подпись, дата)	(И.О. Фамилия)
Руководитель курсовой работы			Назаренко Н.В.
		(подпись, дата)	(И.О. Фамилия)
Консультант			
		(подпись, дата)	(И.О. Фамилия)

2026 г.

РЕФЕРАТ

Расчетно-пояснительная записка 40 с., 9 рис., 14 табл., 1 прил., 27 источн.

ДОСУГ, МЕРОПРИЯТИЕ, БАЗА ДАННЫХ, РЕЛЯЦИОННАЯ МОДЕЛЬ, СИСТЕМА УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ, POSTGRESQL, ТРИГГЕР.

Цель работы – спроектировать и реализовать базу данных для поиска и организации досуга и приложение для взаимодействия с ней.

В аналитической части рассмотрены задачи поиска мероприятий и организации совместного досуга, выполнено сравнение известных решений. Сформулированы функциональные и нефункциональные требования, выделены сущности и связи между ними, определены роли пользователей. По результатам сравнения моделей данных обоснован выбор реляционной модели.

В конструкторской части разработана структура БД для хранения пользователей, мероприятий, мест проведения, категорий, изображений, участия и отзывов. Определены ограничения целостности, проверено соответствие третьей нормальной форме. Разработаны ролевая модель, алгоритм триггера сохранения фактов удаления объявлений и архитектура приложения.

В технологической части обоснован выбор PostgreSQL и средств разработки приложения JoinUp. Реализованы серверная и клиентская части, ограничения БД, разграничение доступа и триггер. Описаны методы тестирования и результаты проверки создания мероприятия, отдельных операций приложения, ограничений целостности, прав БД и триггерной функции.

В исследовательской части определена зависимость времени выполнения запроса поиска по городу и диапазону дат от количества мероприятий с индексом и без него. Для 10000 мероприятий среднее время составило 4,905 мс без индекса и 3,038 мс с ним. В условиях проведенного исследования индекс сократил время выполнения запроса примерно на 38%.

СОДЕРЖАНИЕ

РЕФЕРАТ **3**

ВВЕДЕНИЕ **5**

1 Аналитическая часть **6**
1.1 Анализ предметной области 6
1.2 Анализ известных решений 6
1.3 Требования к программному обеспечению 8
1.4 Формализация данных 9
1.5 Формализация ролей и типов 11
1.6 Анализ моделей баз данных 12
1.7 Формализация задачи 14

2 Конструкторская часть **15**
2.1 Описание сущностей базы данных 15
2.2 Ограничения целостности 19
2.3 Проверка соответствия третьей нормальной форме 19
2.4 Ролевая модель 21
2.5 Используемый триггер 22
2.6 Архитектура приложения 22
2.7 Вывод 23

3 Технологическая часть **24**
3.1 Выбор системы управления базами данных 24
3.2 Выбор средств реализации приложения 24
3.3 Структуры приложения и базы данных 24
3.4 Интерфейс приложения 25
3.5 Тестирование 27
3.6 Вывод 28

4 Исследовательская часть **29**
4.1 Технические характеристики 29
4.2 Постановка исследования 29
4.3 Определение зависимости времени поиска от количества мероприятий 30
4.4 Вывод 31

ЗАКЛЮЧЕНИЕ **33**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **34**

ПРИЛОЖЕНИЕ А. Сценарий создания объектов базы данных **36**

ВВЕДЕНИЕ

Для организации совместного досуга необходимо хранить сведения о мероприятиях, времени и месте проведения, участниках и отзывах. Размещение объявлений в общей ленте затрудняет поиск по заданным условиям и учет участников. Эти задачи решает специализированное приложение с базой данных.

Цель работы – спроектировать и реализовать базу данных для поиска и организации досуга и приложение для взаимодействия с ней.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1) проанализировать предметную область и известные решения;
2) определить требования, сущности и связи между ними;
3) спроектировать БД, ограничения целостности, роли и триггер сохранения истории удаления объявлений, проверить соответствие БД третьей нормальной форме;
4) выбрать средства реализации и разработать приложение;
5) описать методы тестирования и проверить корректность основных операций;
6) исследовать время поиска мероприятий по городу и диапазону дат с индексом по этим полям и без него.

1 Аналитическая часть

В данной части рассматриваются предметная область и известные решения для поиска и организации досуга. Формулируются требования к приложению, определяются состав данных и роли пользователей. Выполняется сравнение моделей баз данных и формализуется задача разработки.

1.1 Анализ предметной области

Под совместным досугом в данной работе понимается участие людей в мероприятиях по общим интересам: прогулках, настольных играх, спортивных встречах и других занятиях. Мероприятие объединяет описание занятия, время, место проведения, организатора и пользователей, выразивших желание участвовать. Например, при поиске компании для настольной игры нужно выбрать подходящий город и дату, узнать место встречи и сообщить организатору об участии.

Поиск мероприятий связан с выбором тематики, места и времени проведения. Такой набор условий используется в специализированных для поиска мероприятий сервисах. Например, Meetup, Mobilizon позволяют искать встречи по интересам, месту, дате и времени [1]. При размещении объявлений в социальных сетях, пользователь вынужден просматривать несколько групп, каналов и прочих источников. Поиск по словам в тексте публикации нельзя назвать удобным, а также он не гарантирует отбор по дате самого мероприятия: дата публикации и дата встречи могут различаться.

Для организации мероприятия в первую очередь необходимо составить его описание, определить его тематику, дату и место проведения. Также необходимо вести учет участников - поддерживать список людей, которые изъявили желание посетить мероприятие. При организации мероприятия через обычную публикацию в социальных сетях вести учет участников проблематично, так как необходимо вручную отслеживать тех, кто собирается посетить мероприятие. Отдельная запись об участии позволяет получить состав участников без просмотра всей переписки и ручной модерации.

Таким образом, основные задачи предметной области – хранение актуальных сведений о мероприятиях, поиск по нескольким условиям, публикация собственных объявлений и ведение списка участников. Отзывы о мероприятиях дополняют эти сведения и позволяют сформировать оценку организатора. Для решения перечисленных задач требуется связанное хранение объявлений, пользователей и их действий.

1.2 Анализ известных решений

Рассмотрим следующие существующие решения для поиска и организации мероприятий:

1) Meetup – сервис поиска встреч по интересам с фильтрами поиска и записью участников [1];
2) Mobilizon – приложение для создания и поиска мероприятий; поддерживает запись без учетной записи по выбору организатора [2];

3) Timepad – сервис публикации мероприятий, регистрации участников и продажи билетов [3];

4) группы ВКонтакте – размещение обычных публикаций о встречах в ленте сообщества с обсуждением в комментариях; отдельные сервисы мероприятий и внешние формы записи в этом варианте не рассматриваются.

Для сравнения выбраны критерии, связанные с поиском и публикацией встреч. Универсальность означает возможность размещать мероприятия разных тематик и городов без заранее заданного ограничения сообщества. Фильтры поиска – отбор по месту, тематике и времени мероприятия. Интеграция с соцсетями означает возможность распространить объявление или организовать общение средствами соцсетей и мессенджеров. Запись без предварительного аккаунта оценивает необходимость отдельной регистрации в сервисе перед записью на мероприятие.

Комбинирование категорий оценивается как возможность назначить мероприятию несколько тематических признаков в отдельных полях карточки: категорий, тем или меток. Это позволяет описывать встречи на пересечении интересов. Хештеги в тексте обычной публикации не приравниваются к отдельным полям карточки мероприятия.

Результаты представлены в таблице 1.1. Доступность в РФ указана по результатам проверки доступа к сайтам из РФ; для Mobilizon результат относится только к проверенному публичному серверу, а не к сервису в целом.

Таблица 1.1 — Результаты сравнения решений

Критерий	**Meetup**	**Mobilizon**	**Timepad**	**Группы ВКонтакте**
Доступность в РФ	Нет	Нет	Да	Да
Интеграция с соцсетями	Да	Да	Да	Да
Универсальность	Да	Да	Да	Нет
Запись без предварительного аккаунта	Нет	Да	Да	Нет
Фильтры поиска	Да	Да	Да	Нет
Несколько категорий или меток мероприятия	Да	Да	Нет	Нет

Meetup позволяет назначить событию до пяти тем, Mobilizon – до десяти меток [1; 2]. В Timepad мероприятие относится только к одной категории, что ограничивает описание встреч на пересечении интересов.

Для групп ВКонтакте универсальность оценена как «нет», поскольку рассматривается отдельное сообщество с заранее заданными рамками. Например, группа «Мероприятия в Москве» ограничена местом проведения, а «Сходки мотолюбителей» – тематикой. Кроме того, обычная лента не предоставляет совместного отбора по городу, категории и времени встречи.

Таким образом, рассмотренные варианты не обеспечивают одновременно доступность из РФ, поиск по месту и времени и описание мероприятий несколькими категориями. На

момент проверки Meetup и выбранный сервер Mobilizon были недоступны из РФ; Timepad ограничивает выбор одной категорией, а группы – рамками сообщества и возможностями ленты. Это обосновывает необходимость разработки сервиса, в котором можно легко создавать и быстро искать нужные мероприятия, используя сочетания тематик.

1.3 Требования к программному обеспечению

На основе сценариев предметной области и результатов сравнения решений определены функциональные требования. Обязательные требования обеспечивают основной цикл работы с мероприятием:

1) регистрация пользователя и вход в приложение;
2) создание объявления с названием, описанием, временем, городом и местом проведения; изменение и удаление своих объявлений;
3) назначение мероприятию нескольких категорий; поиск неудаленных мероприятий по названию, городу, диапазону дат и пересечению выбранных категорий и просмотр сведений о мероприятии без обязательной регистрации;
4) запись пользователя на мероприятие, отказ от участия и просмотр списка участников без повторной записи одного пользователя;
5) изменение и удаление чужих объявлений администратором, сохранение факта удаления объявления в журнале.

Дополнительные требования обеспечивают представление пользователей и обратную связь:

6) просмотр профиля и редактирование собственных данных, просмотр созданных мероприятий и мероприятий с участием пользователя;
7) размещение аватара, обложки мероприятия и ссылки на внешний чат Telegram;
8) просмотр и добавление отзывов с оценкой от 1 до 5 после наступления времени мероприятия, вычисление средней оценки организатора по отзывам.

Приоритет обязательных требований выше дополнительных: без первых невозможно выполнить основной сценарий публикации, поиска и записи, а вторые дополняют его. Отдельная продажа билетов, уведомления и автоматическое подтверждение фактического посещения в задачу не входят.

К нефункциональным требованиям относятся:

— доступ к приложению через браузер без установки отдельной клиентской программы;
— сохранение связности данных: запрет ссылок на отсутствующие сущности и повторения логинов; совместное создание мероприятия и его локации должно выполняться целиком либо отменяться;
— разграничение операций по роли и авторству объявления, проверка прав на сервере;
— разделение интерфейса, обработки запросов и доступа к данным на отдельные компоненты.

Нефункциональные требования обязательны. Производительность поиска оценивается в исследовательской части по времени выполнения запроса при разном количестве мероприятий; заранее заданный норматив времени отклика в рамках работы не устанавливается.

1.4 Формализация данных

Состав данных определяется требованиями к приложению. Для входа и разграничения прав нужна сущность «Пользователь», содержащая учетные и профильные сведения. Центральная сущность «Мероприятие» описывает объявление: название, содержание, время, город и организатора.

Поиск по месту и тематике требует выделения сущностей «Локация» и «Категория». Локация содержит название места, адрес и координаты, категория задает тему мероприятия. Одно мероприятие может относиться к нескольким категориям, а одна категория – объединять разные мероприятия. Для аватаров пользователей и обложек объявлений выделяется сущность «Изображение».

Запись на мероприятие представляется связью «Участие» между пользователем и мероприятием. Один пользователь может участвовать в нескольких мероприятиях, каждое мероприятие может иметь нескольких участников. Поэтому участие и принадлежность мероприятий к категориям образуют связи «многие ко многим». Их отдельное хранение позволяет изменять состав участников и категорий без копирования описания мероприятия.

Для обратной связи нужна сущность «Отзыв», связывающая автора, мероприятие, оценку и текст. Средняя оценка организатора вычисляется по отзывам о его мероприятиях. Для сохранения фактов удаления объявлений выделяется запись журнала удаления, связанная с мероприятием. Полная история редактирования описаний в состав данных не входит.

Мероприятие связано с одним организатором и одной локацией. Организатор и участник являются пользователями: создание отдельных сущностей с повторением их профильных данных не требуется. Выделенные сущности обеспечивают хранение сведений для публикации, поиска, записи, отзывов и контроля удаления объявлений.

На рисунке 1.1 представлена диаграмма БД «сущность-связь» в нотации Чена.

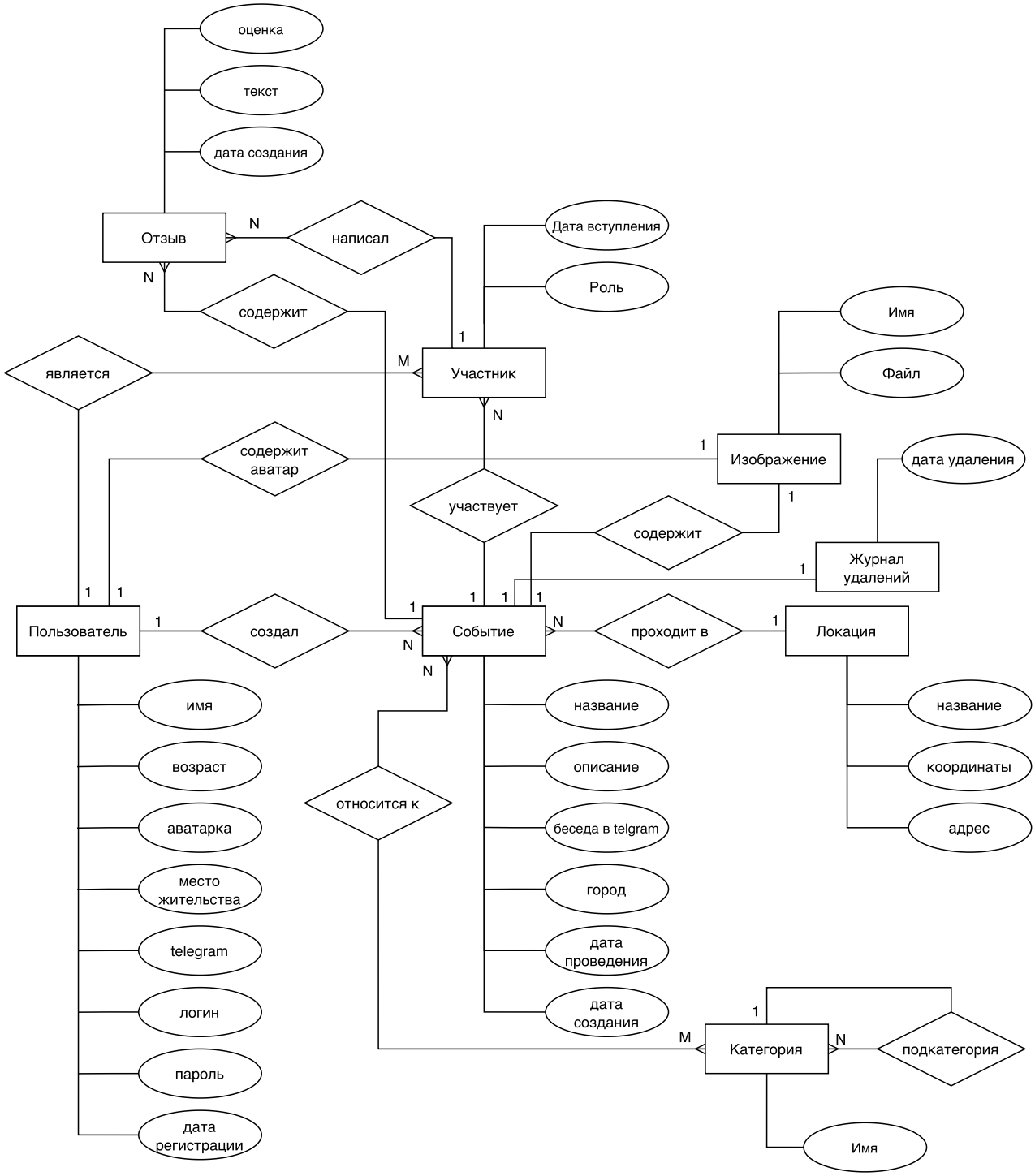


Рисунок 1.1 — Диаграмма «сущность-связь»

1.5 Формализация ролей и типов

Выделены три роли: гость, пользователь и администратор. Гость ищет и просматривает мероприятия без регистрации, чтобы ознакомиться с интерфейсом и содержанием и оценить, подходит ли ему сервис. Роль пользователя предполагает идентификацию, необходимую для связи учетной записи с объявлениями, участием и отзывами и ограничения изменений собственными данными. Администратор наследует действия пользователя и получает право изменять и удалять чужие объявления для ручной модерации.

Организатор – автор конкретного мероприятия. Отдельная постоянная роль для него не нужна: пользователь может организовать собственную встречу и участвовать в чужой. Действия представлены на рисунке 1.2.

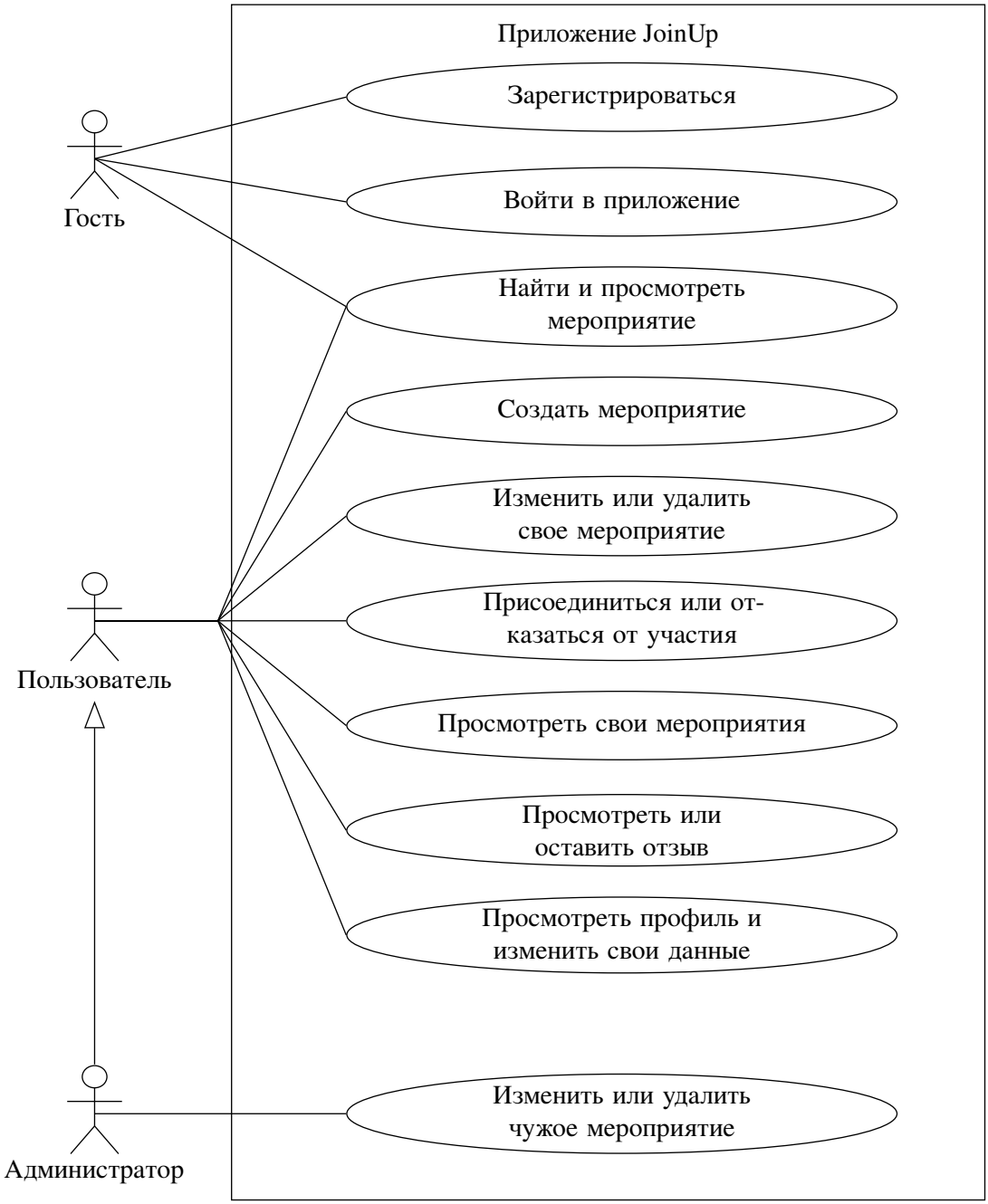


Рисунок 1.2 — Диаграмма прецедентов

1.6 Анализ моделей баз данных

Для приложения необходимы хранение связанных сущностей, изменение отдельных записей и поиск по сочетанию условий. Основная нагрузка связана с оперативной обработкой действий пользователей: публикацией объявления, записью на мероприятие и добавлением отзыва. При выборе модели важны представление связей «многие ко многим», изменение профиля без копирования его данных в каждое мероприятие и выполнение выборок по связанным сущностям.

Рассмотрим дореляционные модели, реляционную модель и основные нереляционные модели. Сравнивается способ организации данных применительно к требованиям приложения; выбор конкретной СУБД выполняется в технологической части.

1.6.1 Дореляционные модели

Иерархическая модель организует записи в дерево, в котором у подчиненной записи один родитель [4]. Она подходит для представления подкатегорий. Однако пользователь может участвовать в нескольких мероприятиях, а мероприятие – относиться к нескольким категориям. Размещение этих данных в одном дереве потребует дублирования записей либо дополнительных связей за пределами основной иерархии.

Сетевая модель допускает несколько связей записи с другими записями. Участие можно представить связями между пользователями и мероприятиями без повторения профилей. Доступ к данным при этом строится вокруг навигации по заданным связям. Для рассматриваемой задачи требуется часто менять сочетание условий поиска, поэтому описание маршрутов обхода менее соответствует этим запросам, чем декларативный отбор связанных записей.

1.6.2 Реляционная модель

В реляционной модели данные представлены отношениями, а связи задаются через ключи [4]. Пользователи и мероприятия хранятся отдельно; участие и принадлежность к категориям представляются самостоятельными отношениями. Изменение имени пользователя затрагивает его запись и не требует изменения каждого мероприятия.

Поиск по городу, диапазону дат и категориям выражается отбором и соединением отношений. Средняя оценка организатора получается агрегированием отзывов о связанных мероприятиях. Ограничения ключей позволяют задавать допустимость ссылок и уникальность пар «пользователь – мероприятие». Этот подход соответствует заранее определенному составу данных и операциям приложения. Его издержкой является необходимость соединять несколько отношений при получении полной карточки мероприятия.

1.6.3 Нереляционные модели

К основным нереляционным моделям относятся документная, графовая, «ключ – значение» и модель семейств столбцов [5]. Их применимость зависит от структуры данных и преобладающих запросов.

Документная модель позволяет хранить мероприятие вместе с вложенными сведениями либо ссылками на другие документы [6]. Вложение списка участников упрощает чтение одной

карточки, но копирование имен и аватаров потребует согласованного обновления нескольких документов. Использование ссылок устраняет это дублирование, однако возвращает необходимость получать связанные данные отдельно. Для фиксированной структуры пользователей, мероприятий и отзывов гибкость документов не является определяющим преимуществом.

Графовая модель представляет сущности вершинами, а связи – ребрами [5]. Она естественно описывает участие и подходит для поиска цепочек знакомств или рекомендаций по связям между людьми. В поставленной задаче такие обходы не требуются: основными являются отбор мероприятий по атрибутам и получение непосредственных связей. Поэтому преимущества графовой модели не определяют выбор хранилища.

Модель «ключ – значение» обеспечивает доступ к объекту по ключу [5]. Для получения одного мероприятия этого достаточно, но поиск по произвольному сочетанию города, даты и категорий потребует дополнительных структур поиска. Модель семейств столбцов организует данные по ключам строк и семействам столбцов [5]. При проектировании под заранее известные запросы можно подготовить отдельные представления данных, однако для рассматриваемого приложения это усложнит согласование сведений при изменении объявления.

В таблице 1.2 сопоставлены способы реализации двух основных операций: хранения участия без повторения профилей и поиска по связанным данным. Оценки относятся к применению моделей в данной задаче, а не к скорости конкретных СУБД.

Таблица 1.2 — Сравнение моделей данных для приложения

Модель	**Представление участия**	**Поиск по связанным данным**
Иерархическая	Дополнительные ссылки между ветвями дерева	Обход дерева и дополнительных ссылок
Сетевая	Связи между записями пользователей и мероприятий	Навигация по заданным связям
Реляционная	Отдельное отношение с ключами пользователя и мероприятия	Отбор и соединение отношений
Документная	Ссылки на документы пользователей	Отбор документов и получение связанных сведений
Графовая	Ребра между пользователями и мероприятиями	Отбор вершин и переход по ребрам
Ключ – значение	Наборы ключей участников	Дополнительные структуры для отбора по атрибутам
Семейства столбцов	Строки с ключами мероприятия и пользователя	Организация строк и дополнительных представлений под запросы

Для разработки выбрана реляционная модель. Она позволяет хранить сущности без копирования профилей, явно задавать связи и ограничения, выполнять комбинированный поиск и агрегировать оценки. Дореляционные модели сильнее связывают операции с маршрутами доступа, а преимущества рассмотренных нереляционных моделей связаны с гибкими документами,

обходом графа или организацией хранения под отдельные запросы. Эти особенности не являются основными требованиями приложения.

1.7 Формализация задачи

Необходимо разработать реляционную БД и приложение для публикации и поиска мероприятий, учета участия и отзывов. Следует предусмотреть возможность назначения мероприятию нескольких категорий и поиск по всем выбранным категориям одновременно. Требуется обеспечить целостность данных, разграничение прав и запись фактов удаления объявлений триггером. После реализации необходимо описать методы тестирования, проверить корректность основных операций и исследовать зависимость среднего времени поиска от количества мероприятий с индексом по городу и дате проведения и без него.

Вывод

В аналитической части определены основные сценарии поиска и организации совместного досуга, сопоставлены возможности известных решений. Сформулированы требования к приложению для публикации встреч, поиска по сочетанию категорий и учета участия, обоснованы состав данных и выделение трех ролей. Для хранения связанных сущностей, задания ограничений и выполнения комбинированного поиска выбрана реляционная модель. Полученные требования и диаграммы служат основой проектирования БД и приложения.

2 Конструкторская часть

В части описаны структура БД, ограничения, нормализация, права доступа, триггер удаления и архитектура приложения.

2.1 Описание сущностей базы данных

На основе сущностей аналитической части определены таблицы пользователей, мероприятий, локаций, изображений, категорий, участников и отзывов. Связь мероприятий с категориями вынесена в отдельную таблицу. Факты удаления хранятся в журнале.

В таблицах ниже для каждого атрибута отдельно указаны тип, ограничения и назначение. В колонке ограничений также приведены значения по умолчанию. Первичный ключ, в том числе составной, уникален; все входящие в него поля обязательны. Остальные поля обязательны только при явном указании. Для строк ограничение длины в схеме не задано, кроме текста отзыва: не более 2000 символов. Обязательность запрещает отсутствие значения, но сама по себе не запрещает пустую строку.

Таблица 2.1 — Структура таблицы пользователей users

Атрибут	**Тип**	**Ограничения**	**Назначение**
id	Идентификатор	Первичный ключ	Идентификатор пользователя
name	Строка	Обязательное	Имя пользователя
city	Строка	Обязательное	Город проживания
age	Целое число	Обязательное; больше нуля	Возраст
login	Строка	Обязательное; уникальное	Логин для входа
password	Строка	Обязательное	Хеш пароля
created_at	Дата и время	Обязательное	Время регистрации
role	Строка	Обязательное; по умолчанию user	Роль в приложении
telegram_login	Строка	Нет	Имя в Telegram
avatar_id	Идентификатор	Внешний ключ на image(id)	Аватар пользователя

Таблица 2.2 — Структура таблицы мероприятий event

Атрибут	**Тип**	**Ограничения**	**Назначение**
id	Идентификатор	Первичный ключ	Идентификатор мероприятия
name	Строка	Обязательное	Название мероприятия
city	Строка	Обязательное	Город проведения
description	Строка	Нет	Описание мероприятия
created_at	Дата и время	Обязательное	Время создания
updated_at	Дата и время	Обязательное	Время изменения
event_date	Дата и время	Обязательное	Время проведения
telegram_chat_url	Строка	Нет	Ссылка на чат
creator_id	Идентификатор	Обязательное; внешний ключ на users(id)	Организатор
location_id	Идентификатор	Обязательное; внешний ключ на location(id)	Место проведения
image_id	Идентификатор	Внешний ключ на image(id)	Обложка мероприятия
deleted	Логический	Обязательное; по умолчанию ложь	Признак удаления

Город хранится в event как отдельное значение для фильтра поиска, а location содержит название места, полный адрес и координаты. Это позволяет искать по городу без разбора адресной строки и создать общий индекс по городу и дате мероприятия. Соответствие города адресу в текущей схеме не проверяется.

Таблица 2.3 — Структура таблиц локаций и изображений

Таблица	**Атрибут**	**Тип**	**Ограничения**	**Назначение**
location	id	Идентификатор	Первичный ключ	Идентификатор локации
location	name	Строка	Обязательное	Название места
location	address	Строка	Обязательное	Полный адрес
location	longitude	Вещественное число	Обязательное	Долгота
location	latitude	Вещественное число	Обязательное	Широта
image	id	Идентификатор	Первичный ключ	Идентификатор изображения
image	name	Строка	Обязательное	Имя файла
image	data	Двоичные данные	Обязательное	Содержимое файла

Таблица 2.4 — Структура таблиц категорий и их связи с мероприятиями

Таблица	**Атрибут**	**Тип**	**Ограничения**	**Назначение**
category	id	Идентификатор	Первичный ключ	Идентификатор категории
category	name	Строка	Обязательное; уникальное	Название категории
category	subcategory_id	Идентификатор	Внешний ключ на category(id)	Родительская категория
event_category	event_id	Идентификатор	Часть первичного ключа; внешний ключ на event(id)	Связанное мероприятие
event_category	category_id	Идентификатор	Часть первичного ключа; внешний ключ на category(id)	Назначенная категория

Таблица 2.5 — Структура таблиц участия, отзывов и журнала

Таблица	**Атрибут**	**Тип**	**Ограничения**	**Назначение**
member	id	Идентификатор	Первичный ключ	Идентификатор участия
member	user_id	Идентификатор	Обязательное; внешний ключ на users(id); пара с event_id уникальна	Участник
member	event_id	Идентификатор	Обязательное; внешний ключ на event(id); пара с user_id уникальна	Мероприятие участия
member	role	Строка	Обязательное	Роль участника
review	id	Идентификатор	Первичный ключ	Идентификатор отзыва
review	event_id	Идентификатор	Обязательное; внешний ключ на event(id); пара с author_id уникальна	Оцениваемое мероприятие
review	author_id	Идентификатор	Обязательное; внешний ключ на users(id); пара с event_id уникальна	Автор отзыва
review	rating	Целое число	Обязательное; от 1 до 5	Оценка мероприятия
review	text	Строка	Обязательное; до 2000 символов	Текст отзыва
review	created_at	Дата и время	Обязательное; по умолчанию текущее время UTC	Время создания отзыва
event_deletion_log	id	Идентификатор	Первичный ключ	Идентификатор записи журнала
event_deletion_log	event_id	Идентификатор	Обязательное; внешний ключ на event(id)	Удаленное мероприятие
event_deletion_log	deleted_at	Дата и время	Обязательное; по умолчанию текущее время UTC	Время удаления

Диаграмма БД представлена на рисунке 2.1. В таблицах указаны атрибуты и их типы; обозначение NN соответствует обязательным полям. Линии показывают связи по внешним ключам с кратностью «многие к одному». Для необязательных внешних ключей связанная запись может отсутствовать.

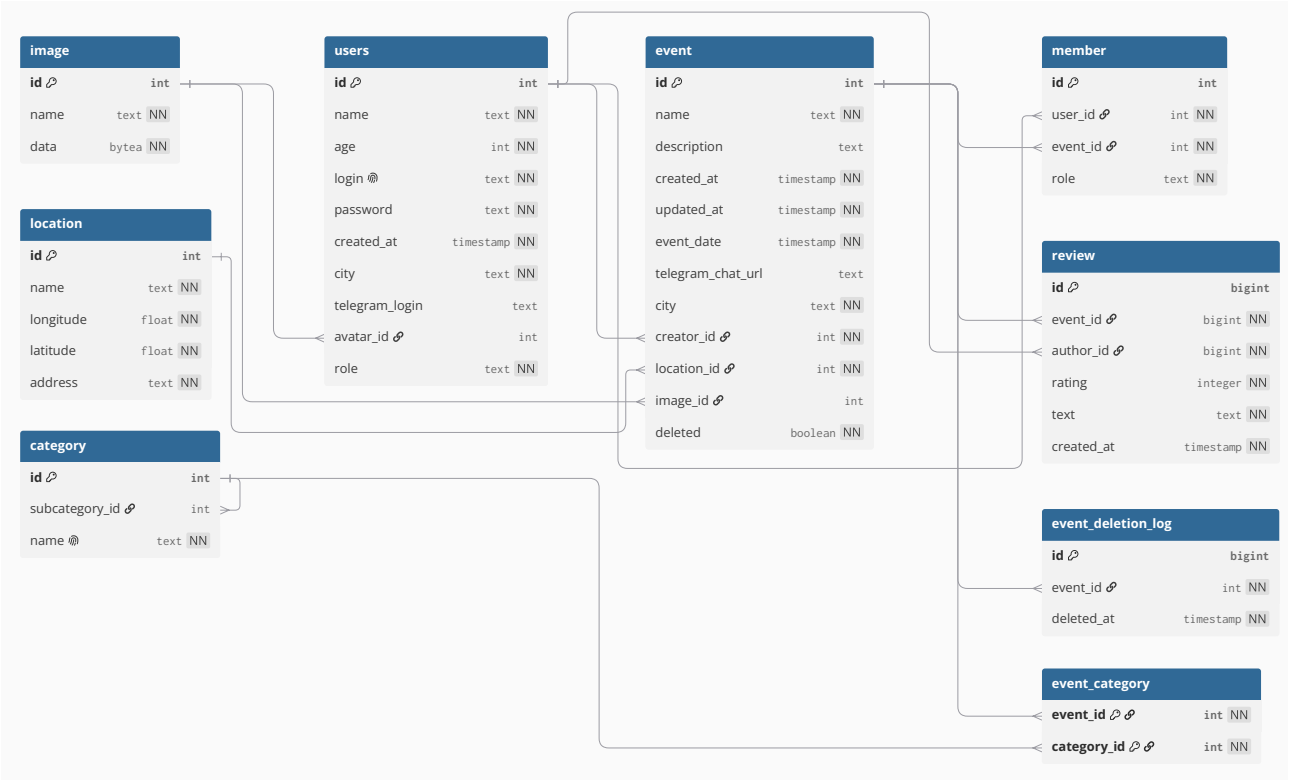


Рисунок 2.1 — Диаграмма базы данных

2.2 Ограничения целостности

Обязательность, уникальность и допустимые значения указаны вместе с атрибутами в подразделе 2.1. Внешние ключи запрещают ссылки на отсутствующие записи.

Удаление объявления изменяет его признак `deleted`; связанные записи сохраняются. При физическом удалении мероприятия каскадно удаляются участие, связи с категориями, отзывы и журнал. Физическое удаление пользователя каскадно удаляет его мероприятия, участия и отзывы. Удаление используемой локации, изображения или категории ограничивается внешними ключами.

Приложение запрещает создание и перенос мероприятия в прошлое, а также редактирование прошедшего мероприятия. Изменять и удалять объявление может автор либо администратор. Отзыв разрешен после завершения мероприятия; его автор не должен быть организатором, а текст после удаления пробелов по краям должен содержать от 1 до 2000 символов.

2.3 Проверка соответствия третьей нормальной форме

Нормализация проверяется по функциональным зависимостям: одинаковые значения определяющих атрибутов должны задавать одинаковые значения зависимых атрибутов. Для

третьей нормальной формы (3НФ) в каждой нетривиальной зависимости определяющий набор должен быть суперключом либо зависимый атрибут должен входить в потенциальный ключ [7].

Потенциальные ключи перечислены в таблице 2.6. Каждый из них определяет остальные атрибуты отношения.

Таблица 2.6 — Потенциальные ключи отношений

Отношение	**Потенциальные ключи**
users	id; login
category	id; name
member	id; (user_id, event_id)
review	id; (event_id, author_id)
event_category	(event_id, category_id)
event, location, image	id в каждом отношении
event_deletion_log	id в каждом отношении

Первая нормальная форма выполняется: каждый атрибут содержит одно значение выбранного домена. Участники мероприятия и его категории представлены отдельными строками отношений member и event_category, а не списками в одной ячейке. Содержимое изображения рассматривается как одно двоичное значение.

В отношениях users, event, location, image, category и event_deletion_log описательные атрибуты зависят от идентификатора записи. Для локации эта зависимость записана в формуле (2.1):

$$\text{id} \to (\text{name}, \text{address}, \text{longitude}, \text{latitude}). \tag{2.1}$$

В users дополнительным потенциальным ключом является login, в category – name. Каждый из них также определяет все остальные атрибуты соответствующего отношения. Все потенциальные ключи перечисленных шести отношений состоят из одного атрибута, поэтому частичные зависимости от ключей отсутствуют.

В отношении member пара пользователь/мероприятие определяет роль участника по формуле (2.2):

$$(\text{user_id}, \text{event_id}) \to \text{role}. \tag{2.2}$$

Один пользователь может иметь разные роли в разных мероприятиях, а участники одного мероприятия – разные роли. Поэтому role зависит от всего составного ключа.

Аналогично для отзыва зависимость от пары мероприятие/автор задана формулой (2.3):

$$(\text{event_id}, \text{author_id}) \to (\text{rating}, \text{text}, \text{created_at}). \tag{2.3}$$

Оценка, текст и время зависят от пары мероприятие/автор: один автор может оставить отзывы о разных мероприятиях, а мероприятие – получить отзывы разных авторов. В member и review составной ключ также определяет id, а id – остальные атрибуты.

В отношении event_category оба атрибута входят в составной первичный ключ, неключевых атрибутов нет. Таким образом, ни в одном из 9 отношений неключевые атрибуты не зависят от части какого-либо потенциального ключа. Условия второй нормальной формы выполнены.

Транзитивные зависимости исключены разделением описательных сведений: event содержит ссылки на организатора, локацию и изображение, а их характеристики хранятся в users, location и image. Аналогично названия категорий находятся только в category; связующие отношения содержат ссылки, журнал – идентификатор мероприятия и время удаления. Рейтинг организатора вычисляется по отзывам.

При проверке приняты следующие ограничения: названия локаций и их координаты не считаются уникальными; город мероприятия задается отдельно от адреса локации, поэтому зависимость location_id $\rightarrow$ city в event не задана. В журнале одному мероприятию могут соответствовать несколько фактов удаления, поэтому event_id не определяет deleted_at.

При принятом наборе функциональных зависимостей неключевые атрибуты определяются потенциальными ключами своих отношений и не зависят транзитивно от них через другие неключевые атрибуты. Следовательно, все 9 отношений разработанной базы данных соответствуют 3НФ.

2.4 Ролевая модель

Приложение поддерживает три подключения к БД, каждое со своей ролью: `joinup_guest`, `joinup_user` и `joinup_admin`. Для обработки запроса выбирается подключение, соответствующее роли пользователя: гость, пользователь или администратор. Матрица доступа приведена в таблице 2.7.

Таблица 2.7 — Права ролей БД

Объекты	**Гость**	**Пользователь**	**Администратор**
event, location, image, category, event_category, review	Чтение	Чтение, добавление, изменение, удаление	Все права на таблицы
users, member, event_deletion_log	Нет	Чтение, добавление, изменение, удаление	Все права на таблицы
Последовательности идентификаторов восьми таблиц с полем id	Нет	Использование и чтение	Все права на последовательности
Подключение к БД, использование схемы public	Да	Да	Да

Полные права администратора относятся к указанным объектам, а не к управлению сервером БД. Принадлежность мероприятия автору и полномочия администратора приложения проверяются перед вызовом репозитория.

Пользовательская роль БД также имеет право добавления в журнал, необходимое триггерной функции. Журнал не защищен от прямого изменения этой ролью. Роль гостя БД имеет

доступ к разрешенным таблицам целиком; фильтрация удаленных мероприятий выполняется приложением.

2.5 Используемый триггер

Триггер `event_deletion_log_trigger` вызывает функцию `log_event_deletion` после обновления признака `deleted` для каждой измененной строки. Вход функции – прежнее и новое состояния строки; результат – запись журнала при переходе признака из ложного значения в истинное.

Схема алгоритма представлена на рисунке 2.2. Повторное присваивание истинного значения не создает запись. Возврат признака в ложное значение также не записывается. При ошибке вставки исходное обновление откатывается вместе с действиями триггера. Журнал содержит идентификатор мероприятия и время удаления, без копии его названия и описания.

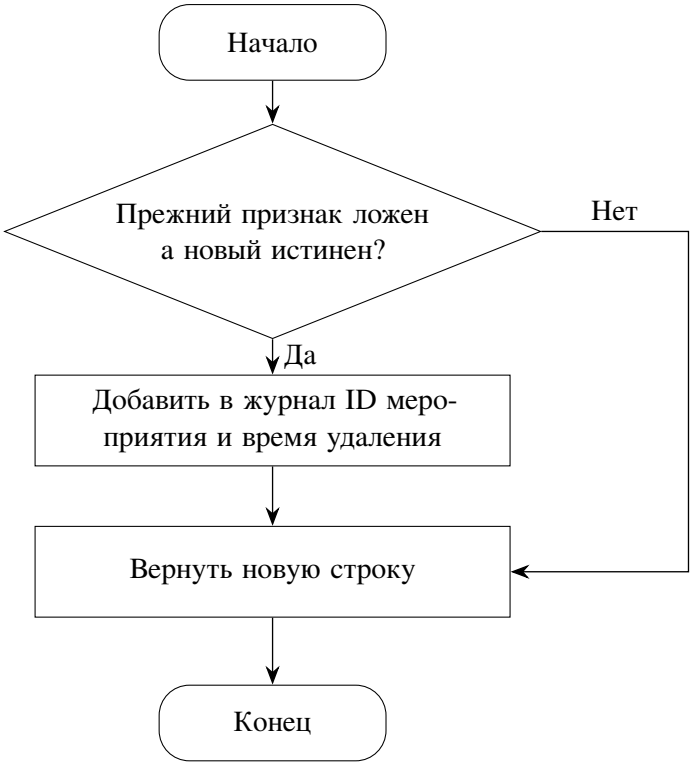


Рисунок 2.2 — Схема алгоритма функции записи факта удаления

2.6 Архитектура приложения

Приложение состоит из frontend-части, backend-части и БД. Схема представлена на рисунке 2.3. Контроллеры принимают запросы и формируют ответы, сервисы проверяют правила предметной области, репозитории выполняют запросы к БД. Для выбора и отображения места frontend обращается к API Яндекс Карт [8].

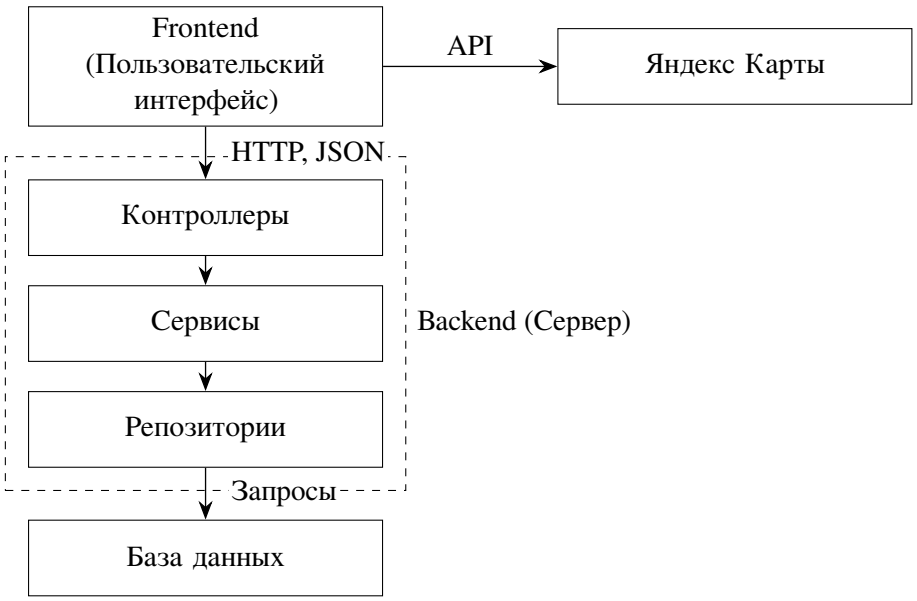


Рисунок 2.3 — Архитектура приложения

Сервер использует параметризованные запросы. Объект управления транзакцией передается репозиториям через контекст операции. Границы транзакций приведены в таблице 2.8.

Таблица 2.8 — Транзакции изменения данных

Операция	**Действия в одной транзакции**
Создание мероприятия	Проверка ссылок, добавление локации и мероприятия
Загрузка обложки	Добавление изображения и изменение ссылки на него в мероприятии
Создание отзыва	Чтение мероприятия, проверка условий и отсутствия отзыва, добавление отзыва
Мягкое удаление	Обновление признака удаления и вставка в журнал триггером в рамках одного оператора

Уникальные ограничения предотвращают повторное участие и повторный отзыв при одновременных запросах. Изменения локации и мероприятия выполняются в разных транзакциях.

2.7 Вывод

Определены структура 9 таблиц, ограничения и права ролей; для принятых зависимостей показано соответствие 3НФ. Разработаны схема триггера, архитектура приложения и границы транзакций.

3 Технологическая часть

В части обоснован выбор средств реализации, описаны БД, интерфейс приложения и результаты тестирования.

3.1 Выбор системы управления базами данных

Для реализации рассмотрены PostgreSQL, MySQL и SQLite. PostgreSQL и MySQL поддерживают внешние ключи, транзакции, построчные триггеры и права на таблицы [9—12]. В SQLite есть триггеры и транзакции [13; 14], но внешние ключи требуют включения, а разграничение прав командами `GRANT` и `REVOKE` отсутствует [15; 16]. Поэтому SQLite не подходит для требуемой ролевой модели.

Выбрана PostgreSQL 16: она поддерживает необходимые ограничения, триггер и роли БД.

3.2 Выбор средств реализации приложения

Сервер написан на Go 1.25 [17]. Для маршрутизации HTTP-запросов используется Echo 5 [18], для доступа к PostgreSQL – pgx/v5 [19], для применения SQL-миграций – Goose 3 [20]. Клиентская часть написана на React 18 [21] и TypeScript [22]; сборку выполняет Vite [23].

HTTP-запросы передаются сервисам и репозиториям по схеме на рисунке 2.3. Авторизация выполняется по токену пользователя.

3.3 Структуры приложения и базы данных

Команда `make migrate_up` применяет SQL-миграции через Goose. Сценарий создания таблиц, ограничений, триггера, индекса и ролей приведен в приложении А.

Идентификаторы созданы типом `serial`, у review и event_deletion_log – `bigserial`. Строковые данные хранятся в `text`, время – в `timestamp`, признак удаления – в `boolean`, изображения – в `bytea`, координаты – в `float`. Ограничения из подраздела 2.1 реализованы первичными и внешними ключами, `UNIQUE`, уникальными индексами и `CHECK`; проверки приложения из подраздела 2.2 выполняются в сервисах.

Для поиска создан частичный индекс `event_search_city_date_idx` по полям `city`, `event_date`, `id` при `deleted = false`. Он соответствует фильтрации по городу и диапазону времени; исследование его влияния приведено в исследовательской части.

Триггер вызывает `log_event_deletion` после обновления поля `deleted`; алгоритм приведен на рисунке 2.2. Миграция ролей назначает права на таблицы и последовательности по матрице из таблицы 2.7. Код функции, триггера и выдачи прав включен в приложение А.

Репозитории выполняют запросы через выбранный пул соединений `pgxpool.Pool`. Значения фильтров и изменяемых полей передаются параметрами `$1`, `$2` и т. д. Границы транзакций приведены в таблице 2.8.

Поиск выполняется методом `SearchEvents`. Запрос исключает удаленные объявления, соединяет `event` с `location`, применяет фильтры названия, дат и города. Для выбора мероприятий, содержащих все указанные категории, применяется подзапрос с группировкой по мероприятию и условием `HAVING COUNT(DISTINCT category_id)`. Результаты упорядочиваются по времени

и идентификатору. После основного запроса сведения об авторах, участниках и категориях загружаются отдельными запросами.

Простой запрос читает пользователя по логину. В `GetUser` агрегатная функция `AVG` вычисляет рейтинг организатора, подзапрос подсчитывает отзывы. Изменяющие запросы добавляют мероприятие и обновляют признак удаления.

3.4 Интерфейс приложения

В клиентском приложении имеются экраны регистрации, входа, поиска, созданных мероприятий, участий, профиля и карточки мероприятия. Поиск с заданными фильтрами показан на рисунке 3.1. В карточке мероприятия (рисунок 3.2) видны описание, место проведения, карта и ссылка на чат. Пользователь может присоединиться к мероприятию, автор – изменить или удалить его. Изображения загружаются через сервер; для выбора адреса и отображения карты используются API Яндекс Карт.

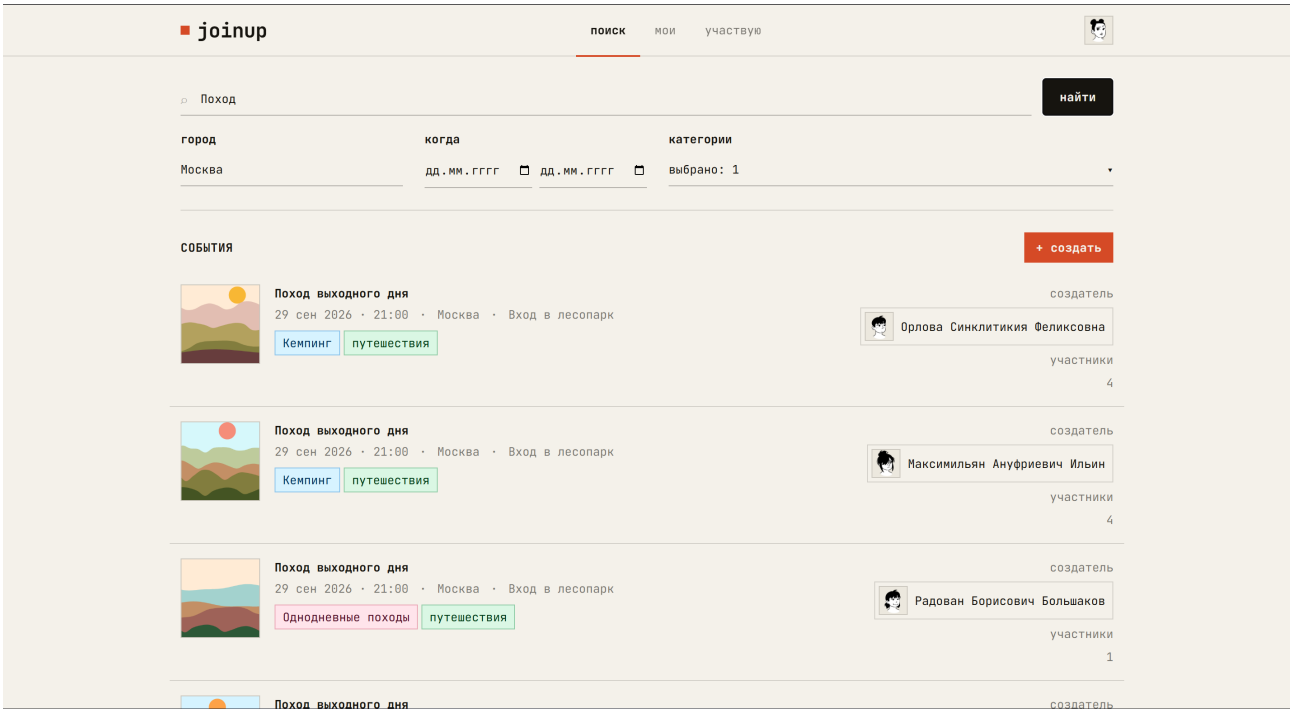


Рисунок 3.1 — Поиск мероприятий с фильтрами

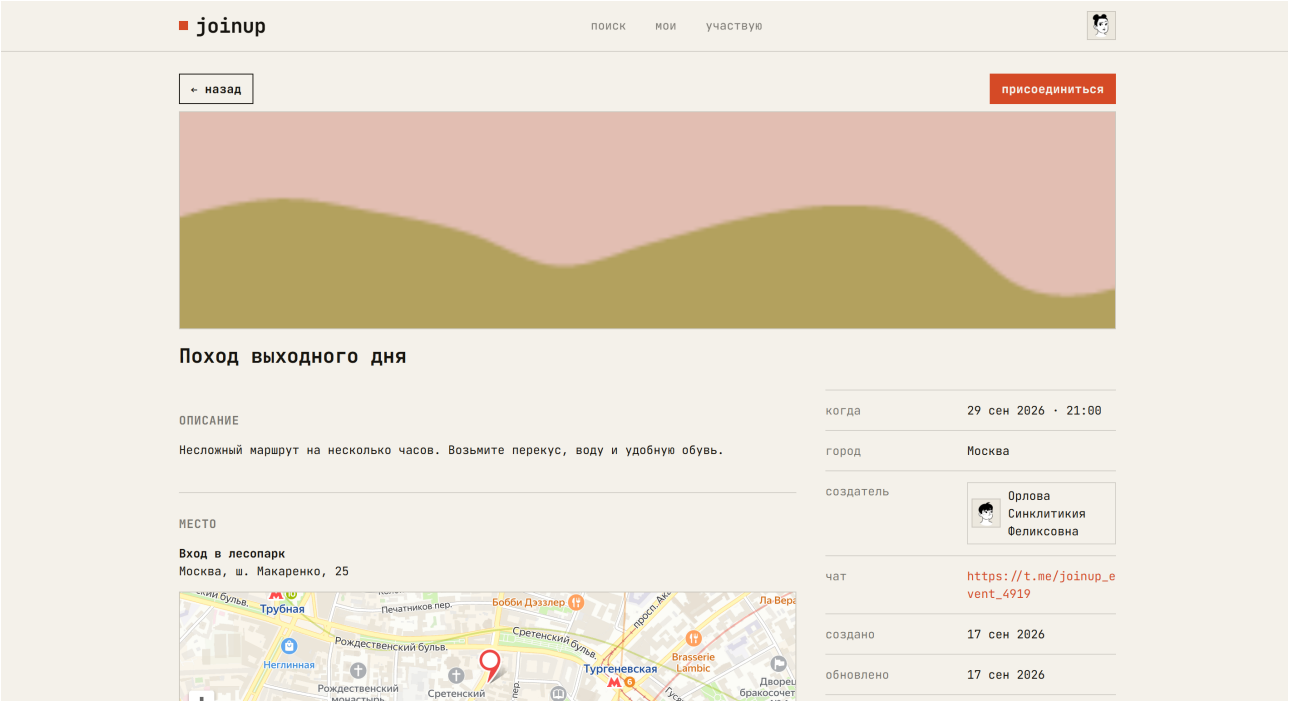


Рисунок 3.2 — Карточка мероприятия

При редактировании автор изменяет название, описание, время и место проведения мероприятия. Профиль с показателями участия и рейтингом представлен на рисунке 3.3.

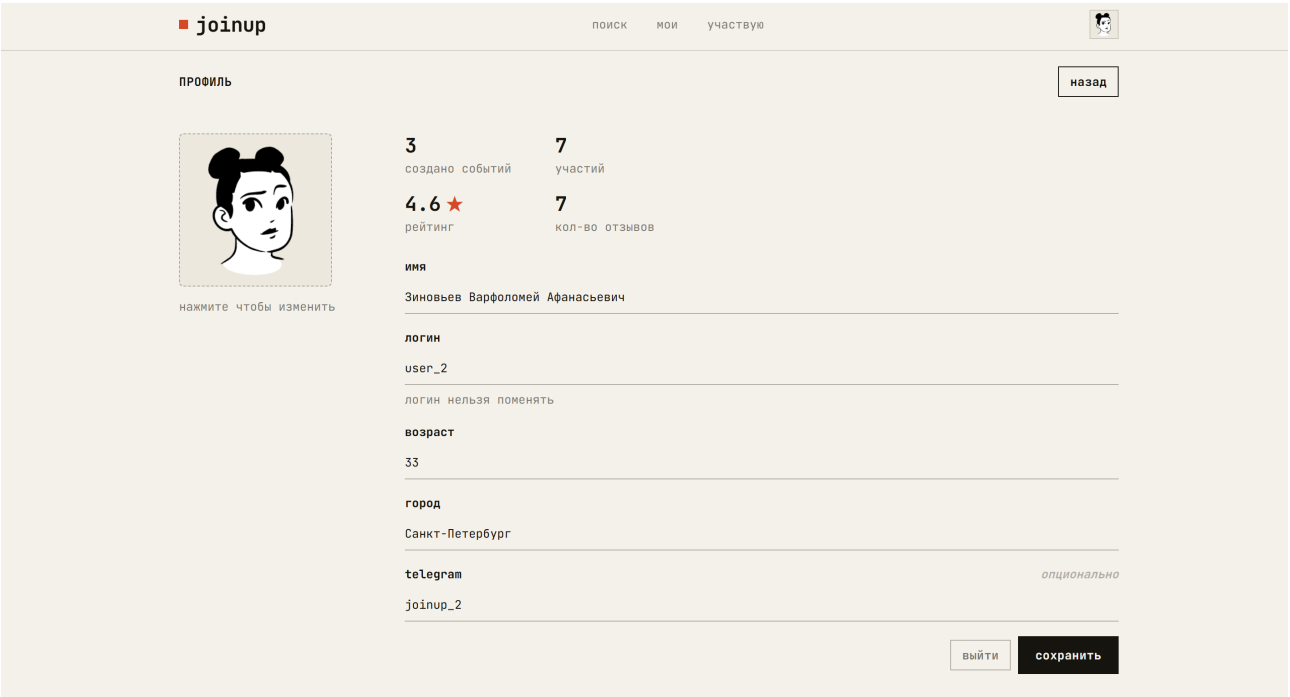


Рисунок 3.3 — Профиль пользователя

Для проверки и демонстрации приложения сценарий app/scripts/seed.py создает искусственные данные: вымышленных пользователей, мероприятия по шаблонам, участия и отзывы. Каждое двадцатое мероприятие помечается удаленным, что также заполняет журнал удаления. Количества строк в использованной БД приведены в таблице 3.1.

Таблица 3.1 — Количество строк в таблицах БД

Таблица	**Строк**	**Таблица**	**Строк**
users	5001	event	10113
location	10113	image	15114
member	32406	category	99
event_category	20224	review	12853
event_deletion_log	505		

3.5 Тестирование

Модульные тесты проверяют бизнес-логику сервисов с подменой зависимостей; интеграционные – работу HTTP-обработчиков и репозиториев с настоящей PostgreSQL. Результаты сравниваются с ожидаемыми ответами и состоянием БД. Модульные тесты выполнены командой `go test ./internal/service`; они завершились успешно.

Четыре интеграционных сценария были выполнены в тестовом контейнере PostgreSQL 16 с миграциями проекта. Перед каждым сценарием таблицы очищались и заполнялись тестовыми данными. Для имитации API-вызовов использовался стандартный пакет httptest [24]; сохраненные мероприятие и локация проверялись SQL-запросом.

Негативные сценарии напрямую проверяли ограничения БД: повтор логина, ссылку на отсутствующую локацию и оценку отзыва вне диапазона. Сверялись код SQLSTATE, имя ограничения и состояние данных после отказа. Результаты приведены в таблице 3.2.

Таблица 3.2 — Проверка создания мероприятия и ограничений БД

Сценарий	**Ожидалось**	**Получено**
Создание мероприятия с новой локацией	HTTP 200; данные мероприятия и локации сохранены	HTTP 200; прочитанные значения совпали с заданными
Повтор логина admin	SQLSTATE 23505; второй пользователь не добавлен	SQLSTATE 23505; в users одна строка
Замена локации на отсутствующую (ID 9999)	SQLSTATE 23503; прежняя связь сохранена	SQLSTATE 23503; мероприятие связано с существующей локацией
Добавление отзыва с оценкой 0	SQLSTATE 23514; отзыв не добавлен	SQLSTATE 23514; в review нет строк

При ручной HTTP-проверке вход с верным паролем возвращал HTTP 200, с неверным – HTTP 401. Поиск по названию «Разговорный клуб английского» и городу Москва вернул HTTP 200 и только неудаленные мероприятия.

Права БД проверены отдельными подключениями: гостю разрешено чтение event и запрещены ее изменение и чтение users; пользователю разрешены чтение и изменение event; администратору – удаление из event. Все шесть результатов совпали с ожидаемыми.

Для функции `log_event_deletion` выделены классы эквивалентности в таблице 3.3. Проверка проведена на мероприятии с `id=1` внутри транзакции с последующим откатом. Указано изменение числа строк журнала для этого мероприятия.

Таблица 3.3 — Классы входных данных триггерной функции

Изменение признака	**Ожидалось**	**Получено**
`false -> true`	+1	+1
`false -> false`	0	0
`true -> true`	0	0
`true -> false`	0	0

После отката транзакции признак удаления и число строк журнала вернулись к исходным значениям.

3.6 Вывод

Реализованы БД PostgreSQL, сервер Go и клиент React. Описаны методы тестирования; проверены создание мероприятия, ограничения БД, права ролей и триггер удаления. Ожидаемые и фактические результаты совпали.

4 Исследовательская часть

В части исследуется влияние индекса по городу и дате на время поиска при разном количестве мероприятий.

4.1 Технические характеристики

Технические характеристики компьютера, на котором проводились измерения:

— операционная система – Windows 11;

— оперативная память – 32 ГБ;

— процессор – AMD Ryzen 7 7700.

PostgreSQL 16.11 работала в WSL 2; для выполнения запросов использовалась библиотека psycopg 3.3.6 [25] на языке Python 3.12.3 [26]. Данные размещены во временных таблицах. Кеш БД перед измерениями прогревался и не сбрасывался. Все неиспользуемые для измерений приложения на компьютере были закрыты. Антивирус отключен.

4.2 Постановка исследования

Цель исследования – определить, как меняется время запроса поиска при наличии индекса по городу и дате и без него для 1000, 2000, 5000 и 10000 мероприятий. Использован запрос метода `SearchEvents`, который соединяет мероприятие с локацией, исключает удаленные объявления, применяет фильтры и сортирует результат по дате и идентификатору. Загрузка категорий, участников и авторов после основного запроса не измерялась.

В обоих вариантах заданы город Москва и интервал от 26.08.2026 до 25.10.2026; фильтры названия и категорий не заданы. Число локаций постоянно и равно 12006; около 5% мероприятий помечены удаленными.

Определение исследуемого индекса приведено в листинге 4.1.

Листинг 4.1 — Индекс для исследования поиска

```
create index research_city_date_idx
    on event (city, event_date, id)
    where deleted = false;
```

Для каждой комбинации объема данных и наличия индекса проведено 10 блоков по 50 измерений. Перед каждым блоком выполнено 10 прогревочных запросов. Порядок вариантов с индексом и без него чередовался между блоками. Таким образом, на каждую точку графика приходится 500 измерений, всего проведено 4000 измерений. Все значения включены в расчет среднего.

Для каждого измерения запрос выполнялся командой `EXPLAIN (ANALYZE, BUFFERS, TIMING OFF, FORMAT JSON)` [27]. Время выполнения на сервере бралось из поля Execution Time в ее результате. Команда также показывает план выполнения запроса – последовательность операций чтения, соединения и сортировки данных – и фактическое число обработанных строк.

Время построения плана выполнения запроса в результат не входит. Для каждого объема данных и каждого варианта (с индексом и без него) среднее вычислено по 500 измерениям времени выполнения одного и того же запроса: 10 блоков по 50 запусков, без учета прогревочных запусков. Погрешность на графике и в таблице представляет половину ширины 95%-го доверительного интервала среднего времени.

При неизвестном стандартном отклонении генеральной совокупности граница погрешности среднего рассчитывается по формуле:

$$\Delta = t_{1-\alpha/2;\,n-1}\frac{s}{\sqrt{n}}, \tag{4.1}$$

где n – число наблюдений, s – их выборочное стандартное отклонение, α – уровень значимости, $t_{1-\alpha/2;\,n-1}$ – квантиль распределения Стьюдента с $n-1$ степенями свободы. Доверительный интервал имеет границы $\bar{x}-\Delta$ и $\bar{x}+\Delta$, где $\bar{x}$ – выборочное среднее.

В исследовании наблюдениями для расчета погрешности служат средние времена десяти блоков, поэтому $n = 10$, а s характеризует разброс этих средних и выражается в миллисекундах. Для доверительной вероятности 0,95 уровень значимости $\alpha = 0{,}05$; при $n-1 = 9$ степенях свободы квантиль $t_{0{,}975;\,9} \approx 2{,}262$. Подстановка в формулу (4.1) дает формулу (4.2):

$$\Delta = 2{,}262\frac{s}{\sqrt{10}}. \tag{4.2}$$

Расчет предполагает независимость и приближенно нормальное распределение средних блоков. Из-за возможной корреляции блоков интервал используется только для ориентировочного сравнения.

4.3 Определение зависимости времени поиска от количества мероприятий

Средние значения замеров приведены в таблице 4.1.

Таблица 4.1 — Время выполнения запроса поиска

Записей	**Без индекса, мс**	**С индексом, мс**	**Найдено**
1000	$0{,}325 \pm 0{,}005$	$0{,}199 \pm 0{,}004$	57
2000	$1{,}824 \pm 0{,}054$	$1{,}583 \pm 0{,}077$	105
5000	$2{,}673 \pm 0{,}080$	$1{,}973 \pm 0{,}043$	287
10000	$4{,}905 \pm 0{,}320$	$3{,}038 \pm 0{,}203$	600

Зависимость времени от числа мероприятий представлена на рисунке 4.1.

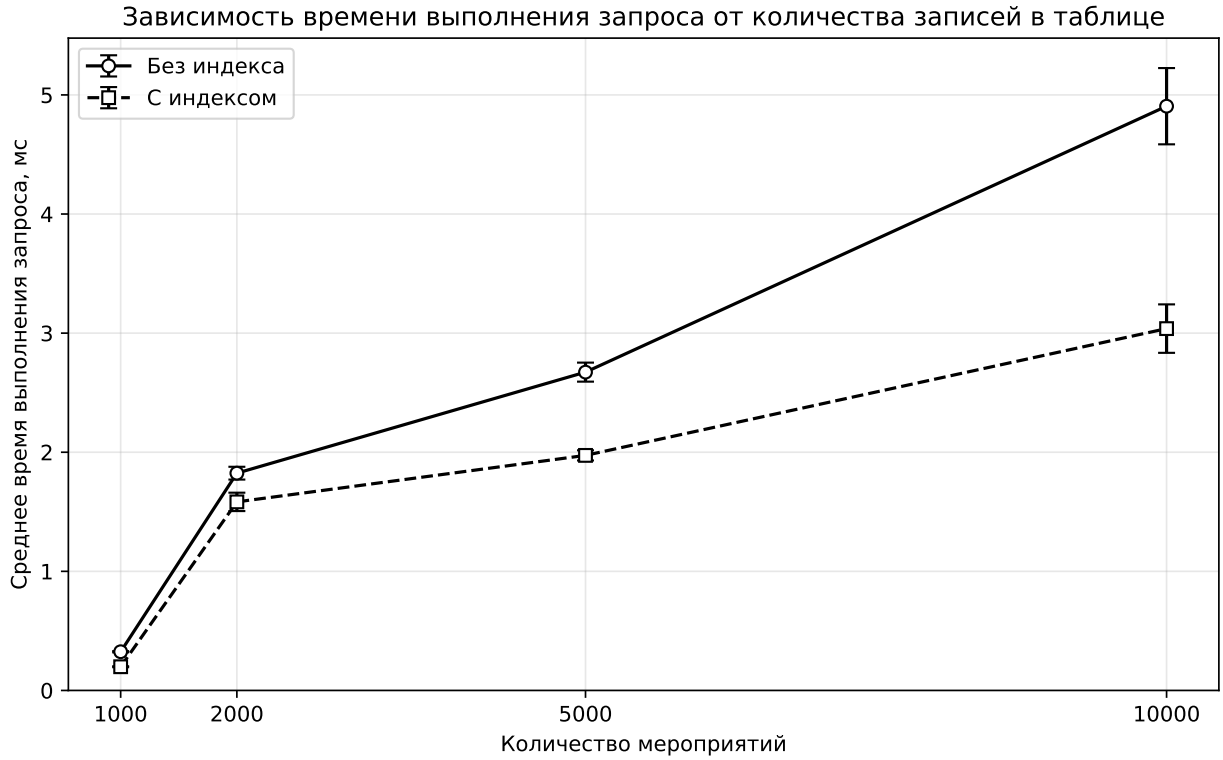


Рисунок 4.1 — Зависимость времени поиска от числа мероприятий

Анализ планов выполнения запроса показывает, что без индекса таблица `event` читается целиком. С индексом операция Bitmap Index Scan находит подходящие записи, а Bitmap Heap Scan читает содержащие их страницы таблицы. При 10000 мероприятий фильтрам соответствуют только 600, поэтому сокращение объема чтения объясняет выигрыш от индекса.

Переход от 1000 к 2000 мероприятий сопровождается резким скачком времени выполнения запроса. Это обусловлено тем, что PostgreSQL меняет способ соединения таблиц `event` и `location`, из которых получает сведения о мероприятии и месте его проведения.

При 1000 мероприятий найдено 57 подходящих. Для каждого PostgreSQL отдельно находит локацию по ее индексу. Это вложенный цикл – Nested Loop.

При 2000 мероприятий найдено 105 подходящих. PostgreSQL создает для них хеш-таблицу – вспомогательную структуру для быстрого сопоставления по идентификатору локации – и просматривает все 12006 локаций. Это хеш-соединение – Hash Join.

Оптимизатор PostgreSQL выбирает способ, который по его оценке потребует меньше работы. При увеличении числа мероприятий эта оценка меняется [27]. В данном случае новый способ требует чтения всех локаций и создания хеш-таблицы. Время каждой из этих операций отдельно не измерялось.

4.4 Вывод

Во всех четырех наборах среднее время с индексом меньше, чем без него. Для 10000 мероприятий оно составило 3,038 мс вместо 4,905 мс, то есть сократилось примерно на 38%. Результат относится к одному SQL-запросу, заданным фильтрам, тестовым данным и прогретому кешу временных таблиц. Он не характеризует время загрузки страницы или работу всего приложения

под нагрузкой, а показывает влияние индекса на скорость выполнения основного поискового запроса внутри приложения.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута: спроектированы и реализованы база данных для поиска и организации досуга и приложение для взаимодействия с ней. Для достижения поставленной цели были выполнены следующие задачи:

1) проанализированы предметная область и известные решения;
2) определены требования, сущности и связи между ними;
3) спроектированы БД, ограничения целостности, роли и триггер сохранения истории удаления объявлений, обосновано соответствие БД третьей нормальной форме;
4) выбраны средства реализации и разработано приложение JoinUp;
5) описаны методы тестирования, проверены создание мероприятия, три ограничения БД, права ролей и работа триггера; результаты совпали с ожидаемыми;
6) исследовано время поиска мероприятий по городу и диапазону дат с индексом по этим полям и без него.

В результате исследования установлено, что во всех рассмотренных наборах данных индекс уменьшает среднее время выполнения основного запроса поиска. При 10000 мероприятий время составило 4,905 мс без индекса и 3,038 мс с ним, то есть сократилось примерно на 38%. Полученный результат относится к условиям проведенного исследования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Meetup [Электронный ресурс]. — URL: https://www.meetup.com/ (дата обращения: 08.10.2026).
2. Mobilizon [Электронный ресурс]. — URL: https://mobilizon.org/ (дата обращения: 08.10.2026).
3. Timepad [Электронный ресурс]. — URL: https://timepad.ru/ (дата обращения: 08.10.2026).
4. What is data modeling? – IBM [Электронный ресурс]. — URL: https://www.ibm.com/think/topics/data-modeling (дата обращения: 24.09.2026).
5. Types of Databases and Database Licenses – IBM [Электронный ресурс]. — URL: https://www.ibm.com/think/topics/database-types (дата обращения: 12.10.2026).
6. Data Modeling in MongoDB – Database Manual [Электронный ресурс]. — URL: https://www.mongodb.com/docs/manual/data-modeling/ (дата обращения: 24.09.2026).
7. Normalization: From Anomalies to Normal Forms – Aalto OpenCS [Электронный ресурс]. — URL: https://opencs.aalto.fi/en/courses/databases/part-8/4-normalization (дата обращения: 25.09.2026).
8. JavaScript API 2.1 – Яндекс Карты [Электронный ресурс]. — URL: https://yandex.ru/dev/jsapi-v2-1/doc/ru/ (дата обращения: 08.10.2026).
9. Supported Features – PostgreSQL 16 Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/16/features-sql-standard.html (дата обращения: 25.09.2026).
10. Privileges – PostgreSQL 16 Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/16/ddl-priv.html (дата обращения: 25.09.2026).
11. Privileges Provided by MySQL – MySQL 8.4 Reference Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/refman/8.4/en/privileges-provided.html (дата обращения: 25.09.2026).
12. InnoDB Transaction Model – MySQL 8.4 Reference Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-model.html (дата обращения: 25.09.2026).
13. CREATE TRIGGER – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/lang_createtrigger.html (дата обращения: 25.09.2026).
14. Transaction – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/lang_transaction.html (дата обращения: 25.09.2026).
15. SQLite Foreign Key Support – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/foreignkeys.html (дата обращения: 25.09.2026).
16. SQL Features That SQLite Does Not Implement – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/omitted.html (дата обращения: 25.09.2026).
17. Documentation – The Go Programming Language [Электронный ресурс]. — URL: https://go.dev/doc/ (дата обращения: 12.10.2026).
18. Quickstart – Echo [Электронный ресурс]. — URL: https://echo.labstack.com/guide/quickstart/ (дата обращения: 12.10.2026).
19. pgx – PostgreSQL Driver and Toolkit for Go [Электронный ресурс]. — URL: https://pkg.go.dev/github.com/jackc/pgx/v5 (дата обращения: 12.10.2026).

20. Overview – Goose [Электронный ресурс]. — URL: https://pressly.github.io/goose/ (дата обращения: 12.10.2026).

21. React Documentation [Электронный ресурс]. — URL: https://react.dev/ (дата обращения: 12.10.2026).

22. TypeScript Documentation [Электронный ресурс]. — URL: https://www.typescriptlang.org/docs/ (дата обращения: 12.10.2026).

23. Getting Started – Vite [Электронный ресурс]. — URL: https://vite.dev/guide/ (дата обращения: 12.10.2026).

24. httptest – Go Packages [Электронный ресурс]. — URL: https://pkg.go.dev/net/http/httptest (дата обращения: 08.10.2026).

25. Psycopg 3 Documentation [Электронный ресурс]. — URL: https://www.psycopg.org/psycopg3/docs/ (дата обращения: 12.10.2026).

26. Python 3.12 Documentation [Электронный ресурс]. — URL: https://docs.python.org/3.12/ (дата обращения: 12.10.2026).

27. Using EXPLAIN – PostgreSQL 16 Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/16/using-explain.html (дата обращения: 12.10.2026).

ПРИЛОЖЕНИЕ А

Команды создания объектов текущей схемы PostgreSQL приведены в листингах А.1–А.7 в порядке их применения.

Листинг А.1 — Основные таблицы

```
create table if not exists image (
    id serial primary key,
    name text not null,
    data bytea not null
);

create table if not exists location (
    id serial primary key,
    name text not null,
    longitude float not null,
    latitude float not null,
    address text not null
);

create table if not exists users (
    id serial primary key,
    name text not null,
    age int not null check (age > 0),
    login text not null,
    password text not null,
    created_at timestamp not null,
    city text not null,
    telegram_login text,
    avatar_id int references image(id)
);

create table if not exists event (
    id serial primary key,
    name text not null,
    description text,
    created_at timestamp not null,
    updated_at timestamp not null,
    event_date timestamp not null,
    telegram_chat_url text,
    city text not null,
    creator_id int not null references users(id) on delete cascade,
    location_id int not null references location(id),
    image_id int references image(id),
    deleted boolean not null default false
);

create table if not exists member (
```

```
    id serial primary key,
    user_id int not null references users(id) on delete cascade,
    event_id int not null references event(id) on delete cascade,
    role text not null
);

create table if not exists category (
    id serial primary key,
    subcategory_id int references category(id),
    name text not null constraint category_name_unique unique
);

create table if not exists event_category (
    event_id int not null references event(id) on delete cascade,
    category_id int not null references category(id),
    primary key (event_id, category_id)
);
```

Листинг А.2 — Ограничение логина и роль пользователя

```
alter table users
    add constraint users_login_unique unique (login),
    add column role text not null default 'user';
```

Листинг А.3 — Категории и уникальность участия

```
create unique index if not exists member_user_event_unique
    on member (user_id, event_id);

insert into category (name)
values
    ('спорт'),
    ('музыка'),
    ('игры'),
    ('арт'),
    ('прогулки'),
    ('учёба'),
    ('еда'),
    ('tech'),
    ('путешествия'),
    ('фото'),
    ('кино')
on conflict (name) do nothing;
```

Листинг А.4 — Отзывы

```
create table if not exists review (
    id          bigserial primary key,
    event_id    bigint    not null
        references event(id) on delete cascade,
```

```
    author_id   bigint    not null
        references users(id) on delete cascade,
    rating      integer   not null
        constraint review_rating_check check (rating between 1 and 5),
    text        text      not null
        constraint review_text_length_check check (char_length(text) <= 2000),
    created_at  timestamp not null default (now() at time zone 'utc'),
    constraint review_event_author_unique unique (event_id, author_id)
);

create index if not exists review_event_id_idx on review (event_id);
create index if not exists review_event_creator_lookup_idx on review (event_id);
```

Листинг A.5 — Журнал и триггер удаления

```
create table if not exists event_deletion_log (
    id bigserial primary key,
    event_id int not null references event(id) on delete cascade,
    deleted_at timestamp not null default (now() at time zone 'utc')
);

-- +goose StatementBegin
create or replace function log_event_deletion()
returns trigger
language plpgsql
as $$
begin
    if old.deleted = false and new.deleted = true then
        insert into event_deletion_log (event_id)
        values (new.id);
    end if;

    return new;
end;
$$;
-- +goose StatementEnd

create trigger event_deletion_log_trigger
after update of deleted on event
for each row
execute function log_event_deletion();
```

Листинг A.6 — Индекс поиска

```
create index if not exists event_search_city_date_idx
    on event (city, event_date, id)
    where deleted = false;
```

Листинг A.7 — Роли и права доступа

```
-- +goose StatementBegin
do $$
begin
    if not exists (select 1 from pg_roles where rolname = 'joinup_guest') then
        create role joinup_guest login password 'guest';
    end if;
    if not exists (select 1 from pg_roles where rolname = 'joinup_user') then
        create role joinup_user login password 'user';
    end if;
    if not exists (select 1 from pg_roles where rolname = 'joinup_admin') then
        create role joinup_admin login password 'admin';
    end if;
    execute format('grant connect on database %I to joinup_guest, joinup_user,
    joinup_admin', current_database());
end;
$$;
-- +goose StatementEnd

grant usage on schema public to joinup_guest, joinup_user, joinup_admin;

revoke all privileges on all tables in schema public from joinup_guest, joinup_user,
    joinup_admin;
revoke all privileges on all sequences in schema public from joinup_guest, joinup_user,
    joinup_admin;

grant select on event, location, image, category, event_category, review to joinup_guest;

grant select, insert, update, delete on
    image,
    location,
    users,
    event,
    member,
    category,
    event_category,
    review,
    event_deletion_log
to joinup_user;

grant usage, select on
    image_id_seq,
    location_id_seq,
    users_id_seq,
    event_id_seq,
    member_id_seq,
    category_id_seq,
    review_id_seq,
    event_deletion_log_id_seq
to joinup_user;
```

```
grant all privileges on
    image,
    location,
    users,
    event,
    member,
    category,
    event_category,
    review,
    event_deletion_log
to joinup_admin;

grant all privileges on
    image_id_seq,
    location_id_seq,
    users_id_seq,
    event_id_seq,
    member_id_seq,
    category_id_seq,
    review_id_seq,
    event_deletion_log_id_seq
to joinup_admin;
```